MEASUREMENT REPORT · SIG API MACHINERY TERRITORY

What actually spikes a kube-apiserver — and what each fix is worth

Relist herds dominate real-world peak load, upstream scale CI never triggers them, and a local storm harness can attribute every recent efficiency change independently — including the ones steady-state benchmarks can't see.

2026-07-17 Harness: 15,000-pod storms, 100-watcher herds, 1s sampling

● **38 claims adversarially verified from raw run data**

CPU UNDER STORM	ALLOCATION RATE	LIST P99	APF REJECTIONS
16.2 cores	**802** MB/s	**2.7** s	**40** / storm
0.06 baseline → restart herd	12 MB/s baseline → 67×	0.20 s unloaded → 13×	0 baseline — on defaults, 15k objects

One `kill -9` of a local apiserver with 100 watchers attached. No 5,000-node cluster required to reproduce the anatomy.

TRIGGERS

Peak load is one family: the relist herd

Watches sever, every reflector re-LISTs the world at once, and the response-assembly cost lands simultaneously. Memory is the sharp edge — API Priority & Fairness bounds concurrency, not per-request memory.

Tier 1 **Fires on every cluster, worst at scale**

- **apiserver restart / rolling upgrade** — watch severance + cold-cache rewarm.
  issue #65954: etcd 100 GB, 4000% CPU
- **Concurrent informer initial LISTs** — controller/kubelet restart waves.
  KEP-3157: ~5× transient memory per LIST
- **Per-node agents with cluster-sized LISTs** — cost scales with node count.
  OpenAI postmortem, Dec 2024

Tier 2 **Episodic but routine**

- **410 "too old" relists** — 75s cache history vs 5min etcd retention. KEP-956
- **Write-burst → watch fan-out** — rollouts, namespace teardown, top-of-minute cron.
  EndpointSlices origin story
- **Cache-bypass reads** — quorum/exact-RV LISTs against etcd. KEP-2340

Tier 3 **Situational amplifiers**

- **CRD / OpenAPI regeneration** · **node-lifecycle eviction cascades** · **exempt-traffic LIST storms** · **post-outage token & cert refresh herds**

The CI gap: clusterloader2's load/density suites drive bounded-QPS sustained load and never sever a watch, restart an apiserver, or cold-start an informer fleet — none of Tier 1 exists in `gce-master-scale-performance`. Peak-memory changes are structurally invisible there.

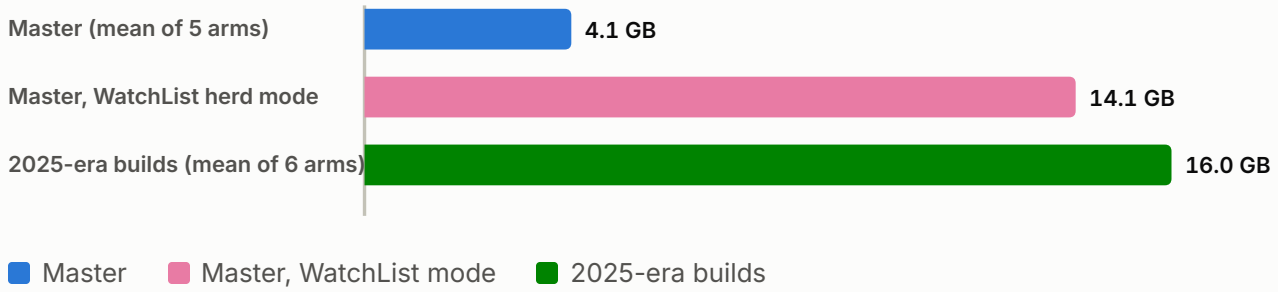
ATTRIBUTION

The same storm, against every build — what each change is worth

Twelve arms, one deterministic scenario, ≥3 reps each: feature-gate flips on one binary, leave-one-out reverts on HEAD, and boundary pairs (`commit^` vs `commit`) where reverts conflict.

Total allocation serving an identical herd

100 watchers × 15,000 pods, herd-connect phase — medians across reps



CHANGE	TECHNIQUE	VERDICT UNDER IDENTICAL STORM
Pooled protobuf allocators #139350	boundary pair	**−66% alloc** · **−30% heap** · **−25% p99** (proto bursts; JSON/Table untouched)
Pod memory-identity conversion #139974 + #139851–#139858	triangulated	**−11.5% of all storm allocation** (→ 0; see below)
WatchList herd mode KEP-3157	herd-mode flip	**−56% restart RSS** · **−20% heap** for **+185–243% total alloc**, **~1.2× CPU**
EtcdRangeStream #136915, on-by-default #140085	gate flip	**2.1× faster rewarm** · **1.8× faster client recovery** (see inversion below)
Presized list results #140200	leave-one-out	**−3% herd alloc** at this shape
ConcurrentWatchObjectDecode #126329	gate flip	warm-time only (−25–33% init at 150k pods); no serving effect at 15k
Buffered proto streaming #140199	leave-one-out	no effect in this scenario — flagged, not claimed
Table-printer allocations #138023	boundary pair	kubectl-side change; server null = methodology validation
Watch-cache lock fixes #139557, #139595	boundary pair + churn	not exercised at this scale; upstream microbench evidence stands

Server-side Table conversion is the heaviest path measured: an 80-LIST Table burst peaks at 9–13GB heap vs 1.3GB for JSON — the strongest peak-framed case for encoding work.

TRIANGULATION

The conversion fix, measured three independent ways

The internal↔versioned pod layout-identity series (instant `unsafe.Pointer` cast) resisted both revert techniques — so it was measured by agreement:

ISOLATED BENCHMARK

24.6 MB → 160 B

core→v1 of a 10k-pod list, run on real v1.36.2 code vs master (60,004 allocs → 3, at every list size)

IN-VIVO PPROF

11.5% → 0%

conversion frames' share of all apiserver allocations under the live storm (1.86 of 16.2 GB); frames absent on master

ARITHMETIC CROSS-CHECK

~1.2 KB/pod ✓

in-vivo cost per pod matches the isolated per-pod cost once object size is accounted for

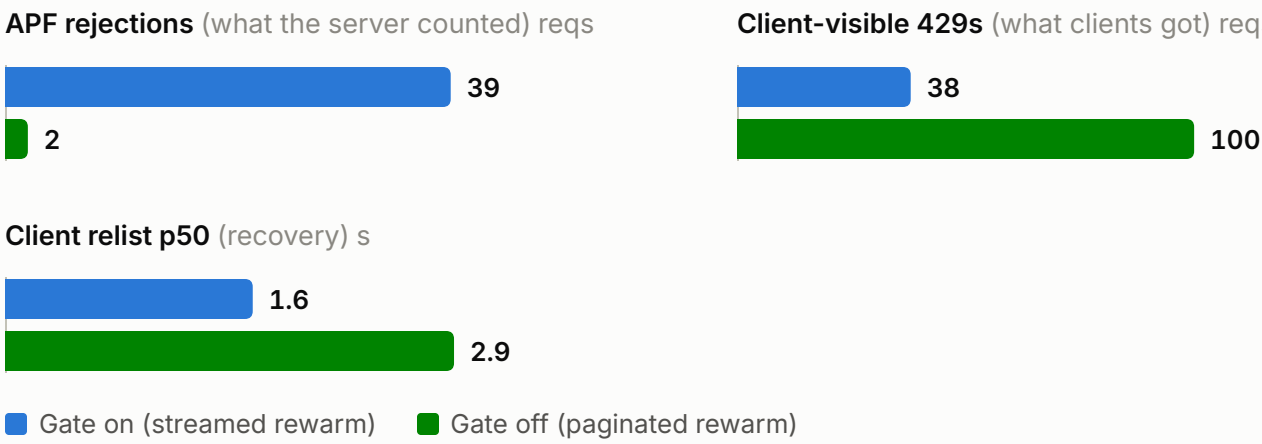
THE INVERSION

Server metrics ranked the worse configuration as better

With EtcdRangeStream off, restart storms *looked* calmer on every server metric. Three hypotheses died against targeted controls; the survivor flipped the verdict — the slower paginated rewarm leaves a longer **cacher-not-ready window**, clients bounce on 429s that never touch the flowcontrol counters, and the retry loop spreads the storm out while clients recover 1.8× slower.

One restart storm, two dashboards

Isolated pair, same binary, only the gate differs — client truth from herd logs

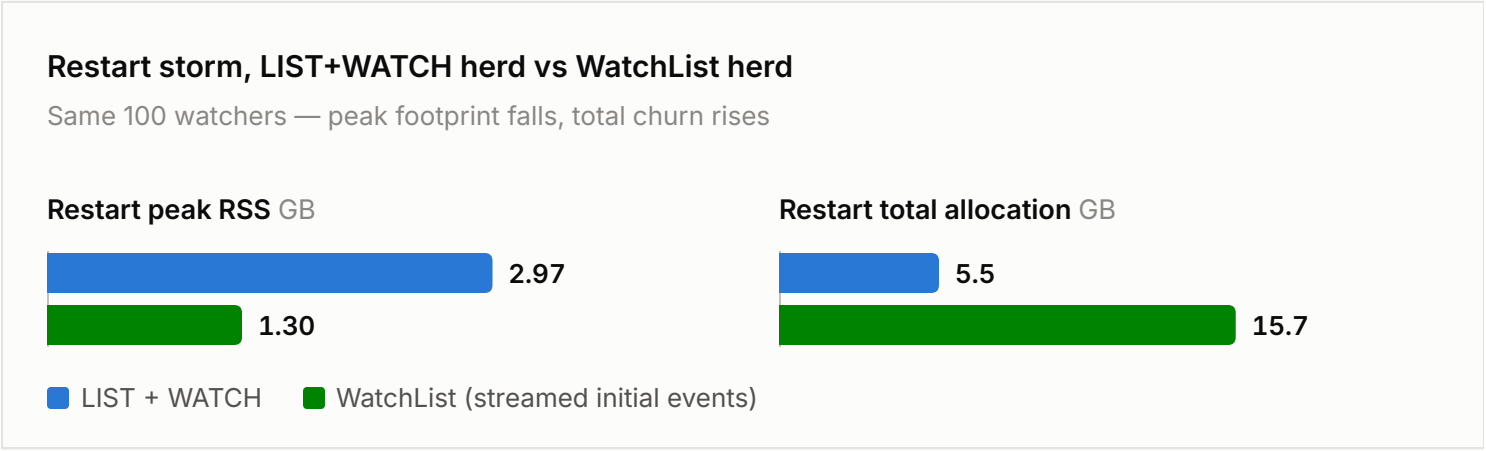


- ✕ **APF size-estimator fed by streamed warm?** Refuted — estimator reads −1 in both arms at +3s and +130s.
- ✕ **Gate changes steady serving?** Refuted — with no herd, 600-LIST bursts match to 0.1% wall-time with indistinguishable CPU profiles.
- ✕ **Not-ready cache returns empty rv=0 lists?** Refuted — all 100 relists in both arms received byte-identical 99.8MB responses.
- ✓ **Not-ready 429 window.** Confirmed — 100 client-visible 429s (2 in APF counters), recovery p50 2.9s vs 1.6s.

Methodology rule this bought: client-observed recovery latency and 429s *by source* are mandatory storm metrics. Server-side phase metrics alone invert verdicts.

TRADE-OFFS

WatchList: buying peak headroom with total allocation



The right trade on memory-constrained control planes; a real CPU/alloc cost where cores are the ceiling.

OPEN QUESTIONS

What would firm up each verdict

Verdicts hold for this scenario (one host, 15k×5KB pods, 100 watchers). Null results are “not exercised,” not “no value” — and no change measured a cost in any phase, so these gaps gate default-flip decisions, not retention.

CHANGE	MISSING DATA	HOW TO GET IT
Memory-identity conversion	write-path / status-churn benefit at scale	alloc-profile diff on a 5k-node scale-CI run
Pooled proto allocators	long-soak pool behavior — bytes *pinned* over hours with mixed response sizes	multi-hour soak with heterogeneous collections
EtcdRangeStream	live storms at 150k+ objects; fleet-scale recovery numbers	bigger seeds + the not-ready-429 metric during real rollouts
WatchList	the CPU↔memory frontier — two points measured (~2.3× CPU on configmaps, ~1.2× on pods); real informer clients	object size/count sweep + a client-go informer herd
Presized lists	scaling curve where slice growth dominates	sweep collection size and selector fraction
ConcurrentWatchObjectDecode	collection-size threshold where it starts paying (150k yes, 15k no)	warm-benchmark sweep over pod counts
Buffered proto streaming	**the workload that exercises it at all** — its designed trigger shape is undetermined here	reproduce the PR's own benchmark shape in the harness
Watch-cache lock fixes	a real contention regime — 300qps churn was too tame for the upstream +43–94% microbench deltas	higher churn × more concurrent LISTs × bigger cache
Table-printer	any client-side measurement — a server harness can't see kubectl code	`kubectl get pods -A` time/alloc harness

INSTRUMENTATION

Making peaks visible where B/op can't see them

A ~50-line patch adds `peak-heap-MB` and `live-heap-after-MB` to `BenchmarkCacherInit` via a 10ms `ReadMemStats` sampler — separating transient peak from resident heap, per iteration, benchstat-trackable.

RANGESTREAM	CONCURRENTDECODE	TIME / INIT	PEAK HEAP	INIT OVERSHOOT	B/OP
off	off	4.5 s	**6.35 GB**	**1.55 GB**	12.5 GB
on	on	**2.0 s**	**5.64 GB**	**0.76 GB**	11.5 GB

150k pods, watch-cache init (the relist path). The peak falls 11% and the overshoot halves while `B/op` moves 8% and `allocs/op` is flat — the alloc integral structurally understates peak-memory changes.

UPSTREAM

Three items ready to carry

1. **Peak-heap `ReportMetric` for BenchmarkCacherInit** — patch in hand; makes streaming-vs-buffering benefits CI-regressable.
2. **A restart-storm phase for clusterloader2** — inject the herd upstream CI currently never sees; the single highest-value addition to scale testing.
3. **Cacher-not-ready 429 observability** — client-visible retry storms are invisible in flowcontrol metrics today; the inversion above is the incident report.

Local harness: seed / herd / burst / churn · 1s envelope sampling · per-phase WSS · pprof at peak

Sources: public upstream (KEPs, issues, postmortems, k8s commits) + local runs — sanitized for sharing